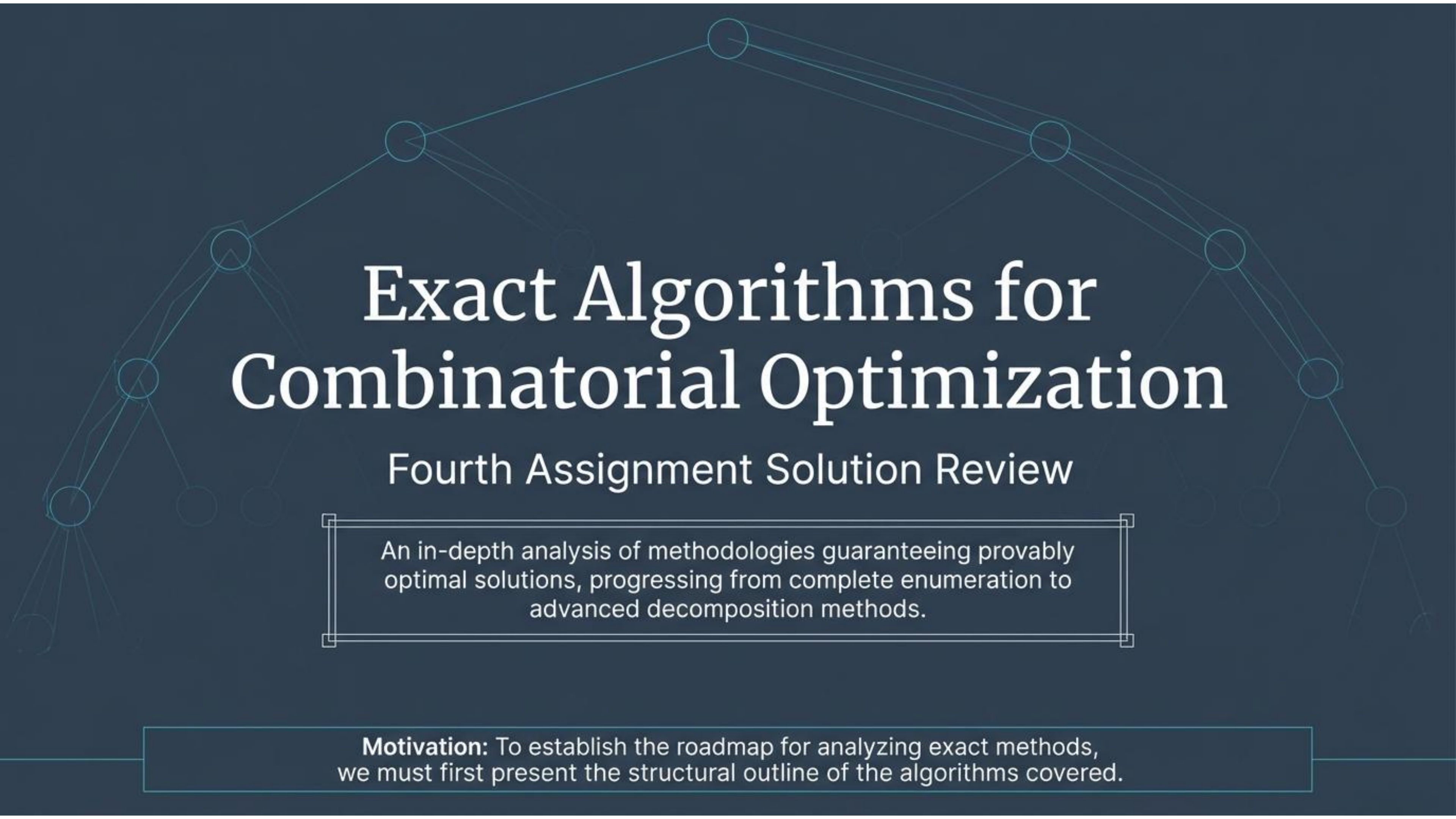




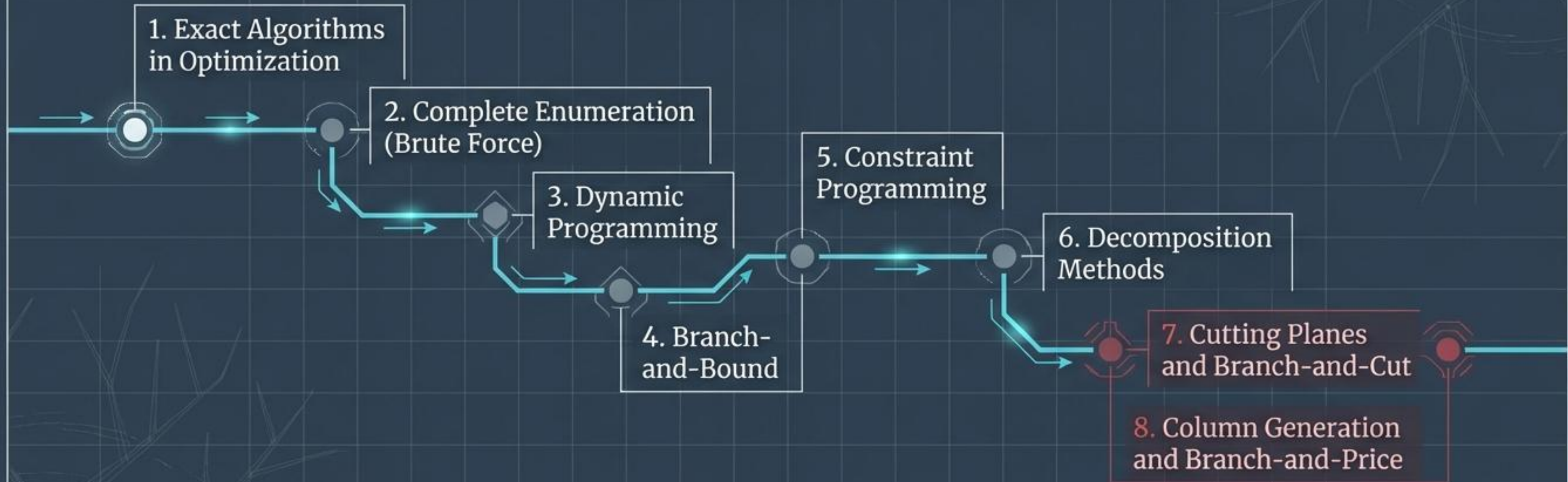
Exact Algorithms for Combinatorial Optimization

Fourth Assignment Solution Review

An in-depth analysis of methodologies guaranteeing provably optimal solutions, progressing from complete enumeration to advanced decomposition methods.

Motivation: To establish the roadmap for analyzing exact methods, we must first present the structural outline of the algorithms covered.

Outline



Motivation: Before exploring specific algorithms, we must rigorously define what makes an algorithm exact and establish the mathematical role of bounds.

[1. Exact Algorithms]

Exact Algorithm:

Guarantees a **provably optimal** solution (or correct yes/no answer) whenever a solution exists, or correctly reports **infeasibility**, typically within finite time for each fixed input instance.

Algorithm Types

Exact	Guarantees optimum	Exponential worst-case complexity	Proves infeasibility
Heuristic	Approximates good solutions	Fast problem-specific rules	No worst-case guarantee
Approximation	Does not attain exact optimum	Polynomial time	Provable performance ratio relative to optimum

Motivation: The most fundamental exact method is to systematically evaluate every candidate. We examine this baseline approach next.

Role of Bounds & Relaxations

Relaxations:

Replace hard constraints with easier ones; optimal value bounds the original problem.

Bounds:

Upper (max) or Lower (min) limits used to prune search trees. Crucial for branch-and-bound and decomposition.

[2. Complete Enumeration] - Algorithmic Blueprint

Definition: Systematically generating and evaluating all feasible solutions.

Complexity: Requires exponential time (e.g., $O(2^n)$ for binary, $O(n!)$ for TSP). Limited to very small instances or benchmark evaluations.

Anatomy: 0/1 Knapsack

Problem: $\max 8x_1 + 6x_2 + 5x_3$

Subject to: $4x_1 + 3x_2 + 2x_3 \leq 5, x_i \in \{0, 1\}$

Objective: Find the binary vector maximizing profit without exceeding capacity.

Components: 3 items, yielding $2^3 = 8$ feasible combinations.

Tuple	Weight	Profit	Status
(0,0,0)	0	0	Feasible
(1,0,0)	4	8	Feasible
(0,1,1)	5	11	Optimal ($z^* = 11$)
(1,1,1)	9	-	Infeasible

Motivation: Exponential growth makes brute force impractical; we must exploit overlapping subproblems to reduce computation time.

[3. Dynamic Programming]

Main Idea: Decomposing problems into overlapping subproblems, solving each once, and storing the solution in a table (memoization/tabulation).

Core Concepts

Principle of Optimality (Bellman)

Remaining decisions must constitute an optimal policy with respect to the state resulting from the first decision.

Suitability

Requires optimal substructure and overlapping subproblems.

Traversal Strategies

Backward DP

References previous stages via classical recurrence. Computes from the end state backwards.

Forward DP

Propagates information forward through reachable states from a starting condition in natural time order.

Motivation: To understand Dynamic Programming mechanically, we must define its exact mathematical recurrence, structural pseudocode, and logical flowchart.

[3. Dynamic Programming] & [9. Algorithm Design]

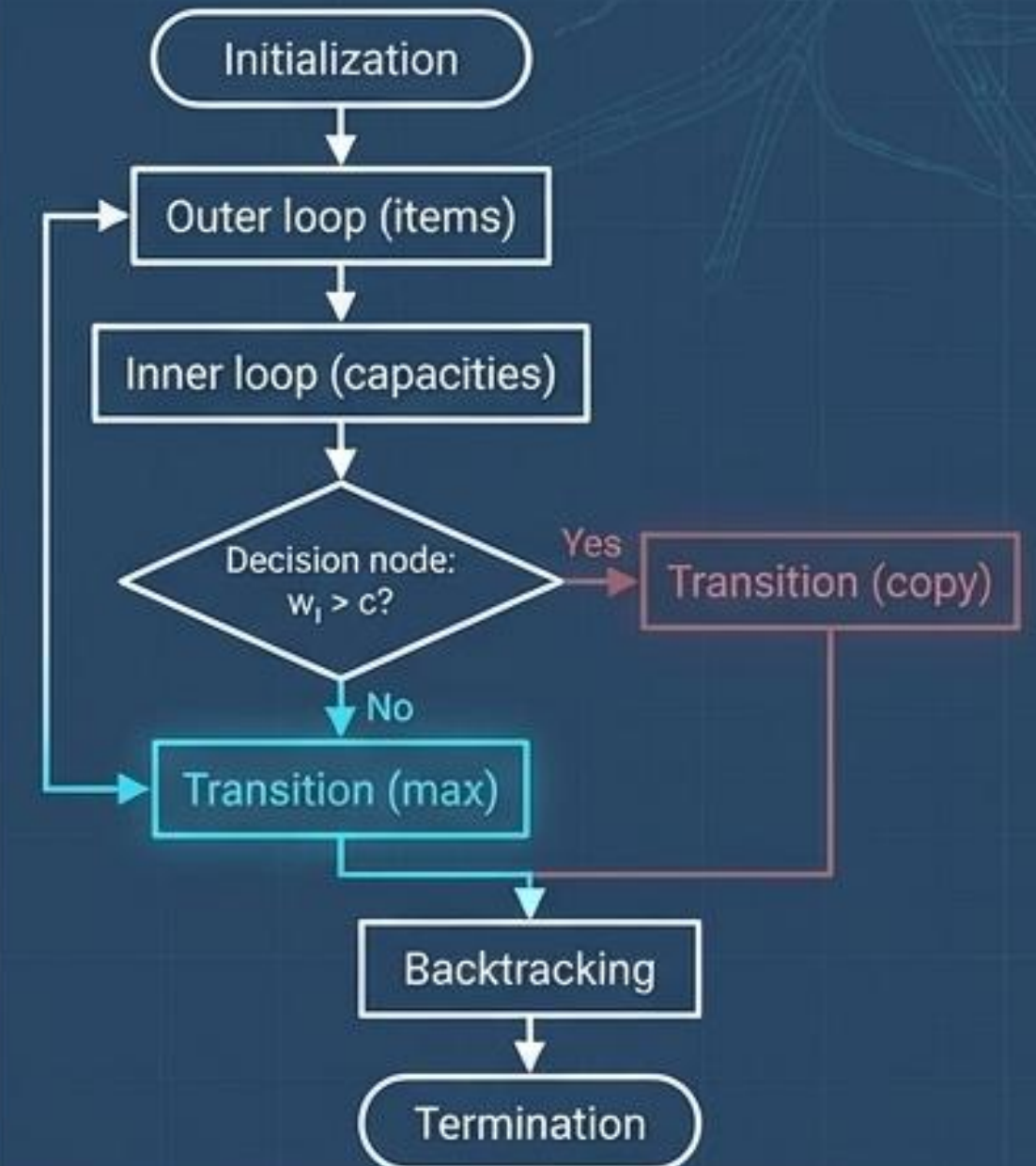
Mathematical Formula

$$\begin{aligned} V(i, c) &= V(i-1, c) && \text{if } w_i > c \\ V(i, c) &= \max\{V(i-1, c), V(i-1, c-w_i) + p_i\} && \text{if } w_i \leq c \end{aligned}$$

Pseudocode

```
1: Create table V[0..n][0..W]
2: for i = 1 to n do
3:   for c = 0 to W do
4:     if wi > c then
5:       V[i][c] ← V[i-1][c]
6:     else
7:       V[i][c] ← max(V[i-1][c], V[i-1][c-wi] + pi)
8:   reconstruction loop...
```

Flowchart Logic



Motivation: With the mathematical and programmatic architecture defined, we now execute this algorithm on a numerical instance.

[3. Dynamic Programming]

Anatomy: Knapsack Step-by-Step

Problem: Profits $p=(6, 4, 5)$, weights $w=(3, 2, 4)$, capacity $W=5$.

Objective: Maximize profit within capacity W .

Components: States (i, c) for items i in $\{0..3\}$, capacities c in $\{0..5\}$.

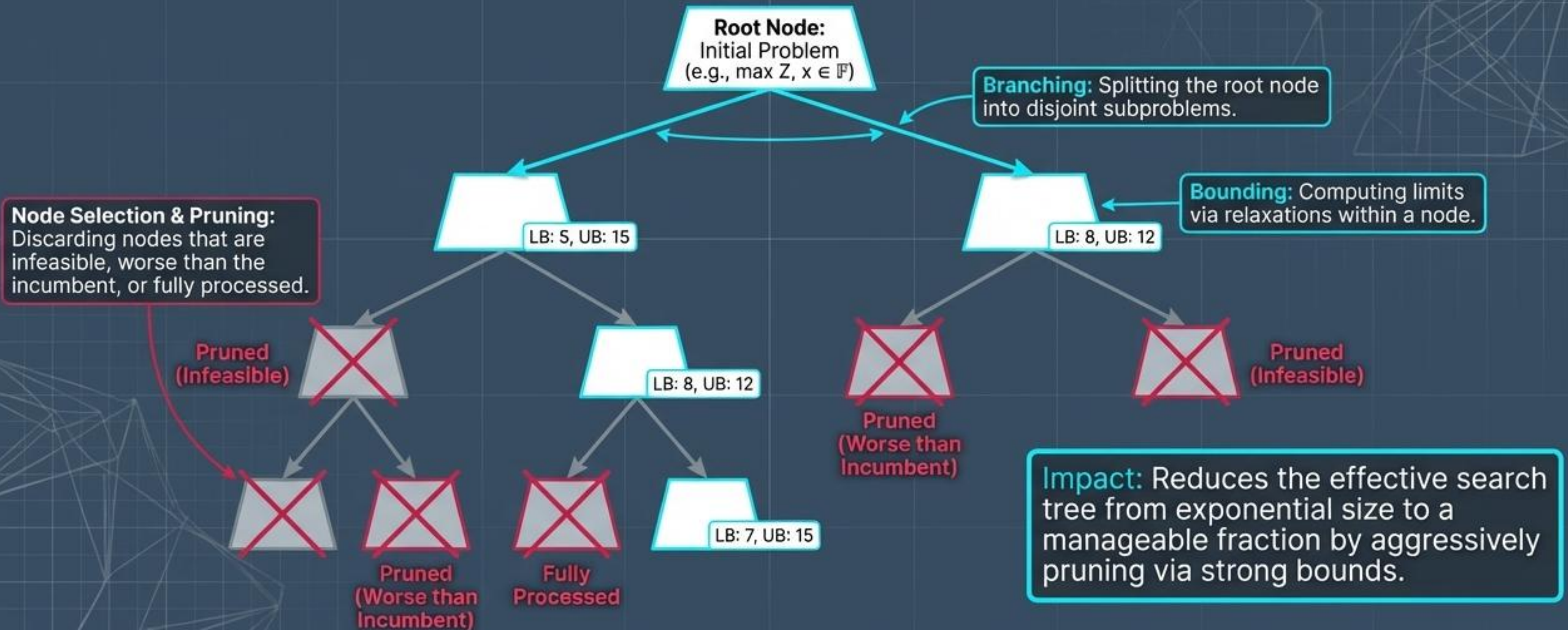
	c=0	c=1	2	3	4	5
i=0	0	0	0	0	0	0
i=1	0	0	0	6	6	6
i=2	0	0	4	6	6	10
i=3	0	0	4	6	6	10

Optimal: $(1, 1, 0)$, $z^* = 10$

Motivation: DP is highly effective but pseudo-polynomial. For large combinatorial spaces lacking dense overlapping structures, we utilize spatial bounding.

[4. Branch-and-Bound]

Main Idea: An exact tree search algorithm that recursively partitions the feasible region and uses relaxation bounds to discard suboptimal subproblems.



Motivation: We will formalize this tree-search methodology mathematically and programmatically before executing it.

Branch-and-Bound: Formalized Tree Search & Bounding

Mathematical Formula

$\max f(\mathbf{x})$ subject to $\mathbf{x}$ in S

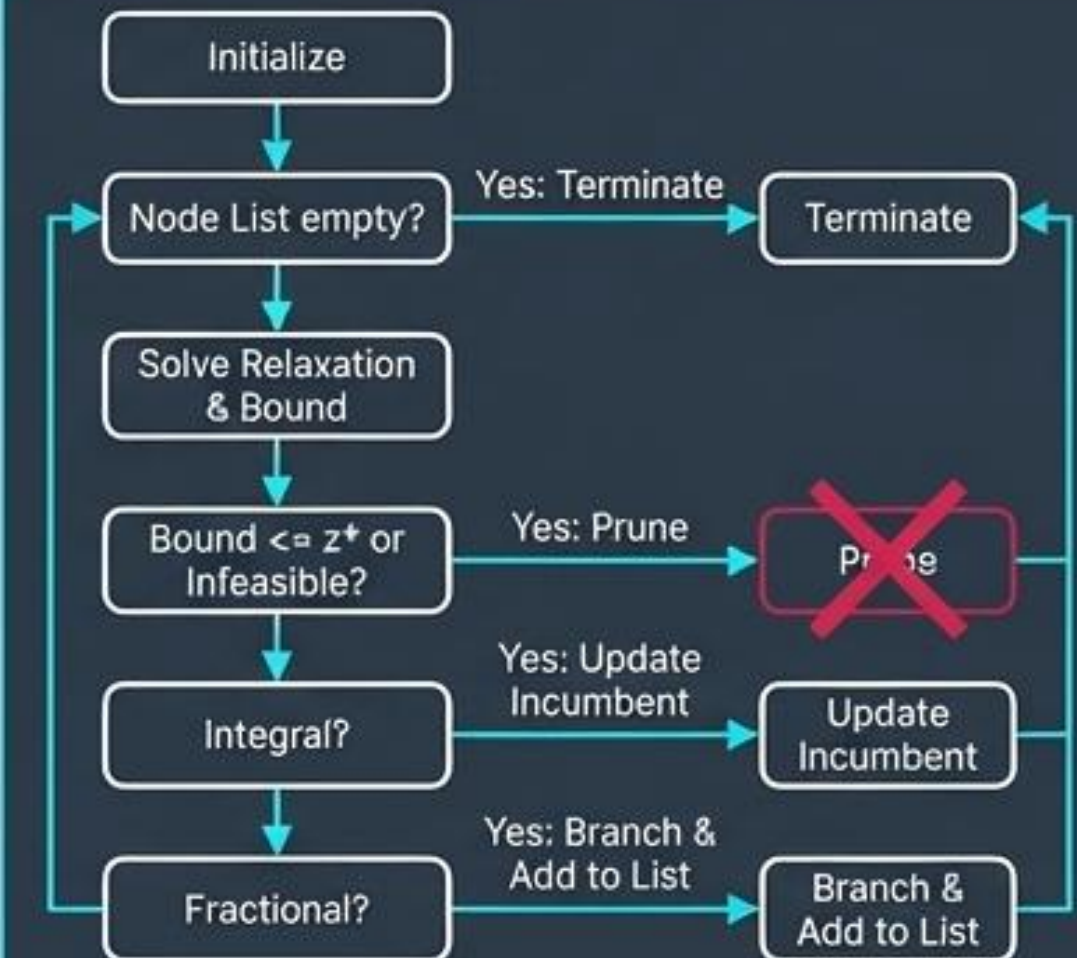
Bound $UB(N)$ computed via continuous relaxation of node N .

Pseudocode

○○○

```
1: Initialize L,  $z^* \leftarrow -\text{infinity}$ 
2: while L is not empty do
3:   Select node N from L
4:   Compute  $UB(N)$ 
5:   if  $UB(N) \leq z^*$  or infeasible then
6:     prune N
7:   else if integral then
8:     update  $z^* \leftarrow f(x_N)$ 
9:   else branch into  $N_1, N_2$ 
```

Flowchart Logic



Motivation: Applying this generic tree search to a Mixed Integer Program reveals how linear relaxations drive the bounding process.

[4. Branch-and-Bound]

Anatomy: Small Integer Program

Problem: $\max 4x_1 + 3x_2$

s.t. $2x_1 + x_2 \leq 7$

$x_1 + 2x_2 \leq 7$

$x_1, x_2 \in \mathbb{Z}_{\geq 0}$

Objective: Find optimal integer point.

Components: Continuous LP relaxations at each tree node.

Incumbent
(Pruned)

~~Node 1a ($x_2 \leq 2$)~~
~~Integer: (2, 2) $z=14$~~

~~Node 1b ($x_2 \geq 3$)~~
~~Max integer value $13 \leq 14$~~

Node 0 (Root)

LP: $(7/3, 7/3)$

Bound: 16.33 (Fractional)

Node 1 ($x_1 \leq 2$)

LP: (2, 2.5)

Bound: 15.5 (Fractional)

Node 2 ($x_1 \geq 3$)

LP: (3, 1)

Integer. $z=15$

Optimal: $x_1^*=3, x_2^*=1, z^*=15$

Motivation: While B&B relies heavily on linear relaxations, many combinatorial problems involve complex logical constraints better suited for constraint propagation.

[5. Constraint Programming]

Main Idea: Stating combinatorial problems as Constraint Satisfaction Problems (CSPs). MIP uses linear inequalities and LP relaxations; CP uses high-level non-linear logical constraints and tailored inference.

Key Components: Global constraints (e.g., `alldifferent`), Symmetry breaking, Constraint propagation, Domain reduction, Backtracking search.

Anatomy: Scheduling A, B, C

Problem: Schedule tasks A, B, C of duration 1 in slots {1, 2, 3}.

Constraints: $A < B, C \neq 1, \text{alldifferent}(A, B, C)$.

Step 1: Initial Domains based on $C \neq 1$

$D(A) = \{1, 2, 3\}$ $D(B) = \{1, 2, 3\}$ $D(C) = \{2, 3\}$

Step 2: Apply $A < B$ (implies $A \neq 3$)

$D(A) = \{1, 2, \text{X}\}$

Step 3: Apply `alldifferent` (forces permutation)

If $C=2 \rightarrow A=1, B=3$ If $C=3 \rightarrow A=1, B=2$

Motivation: CP handles extreme logical complexity, but large-scale structured mathematical models require advanced problem decoupling.

[6. Decomposition Methods]

General Idea: Exploiting the special structure of large optimization problems by splitting them into smaller, manageable subproblems coordinated iteratively.

Benders Decomposition

Mechanism

Adds rows (cuts) to the master problem.

Formulation

$\min c^T \mathbf{x} + \theta$ s.t. Benders cuts.
Master variables $\mathbf{x}$, subproblem $\mathbf{y}$.

Use Case

When complicating variables (often integer) are in the master, and subproblems become easy once $\mathbf{x}$ is fixed.

Dantzig-Wolfe Decomposition

Mechanism

Adds columns (variables) to the restricted master.

Formulation

Expresses $\mathbf{x}^{(k)}$ as convex combination of extreme points $\sum \lambda_{kj} \mathbf{v}_{kj}$.

Use Case

LPs with block diagonal structure; natural representation by patterns or routes.

Motivation: To dynamically strengthen relaxations during the branch-and-bound tree search, we utilize the row-addition concept via cutting planes.

[7. Cutting Planes and Branch-and-Cut]

Cutting Plane

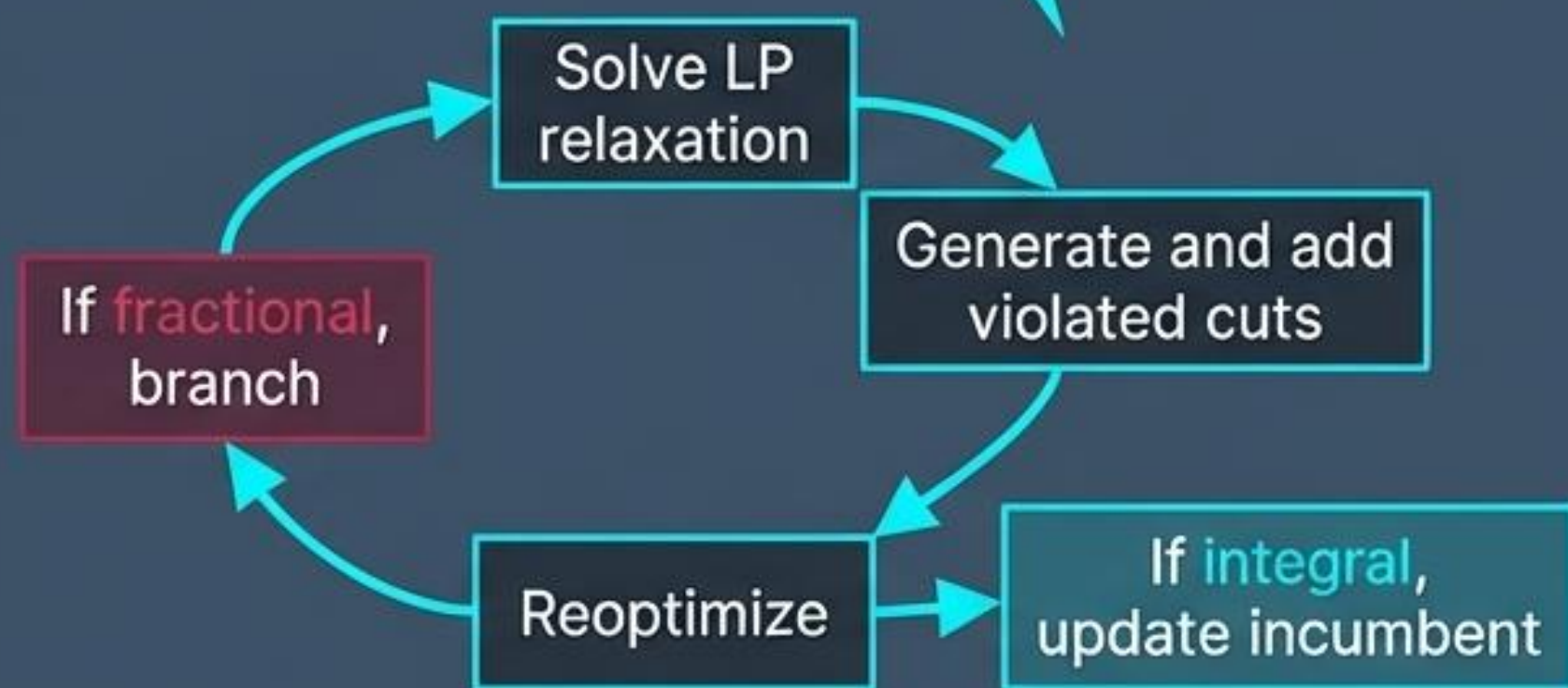
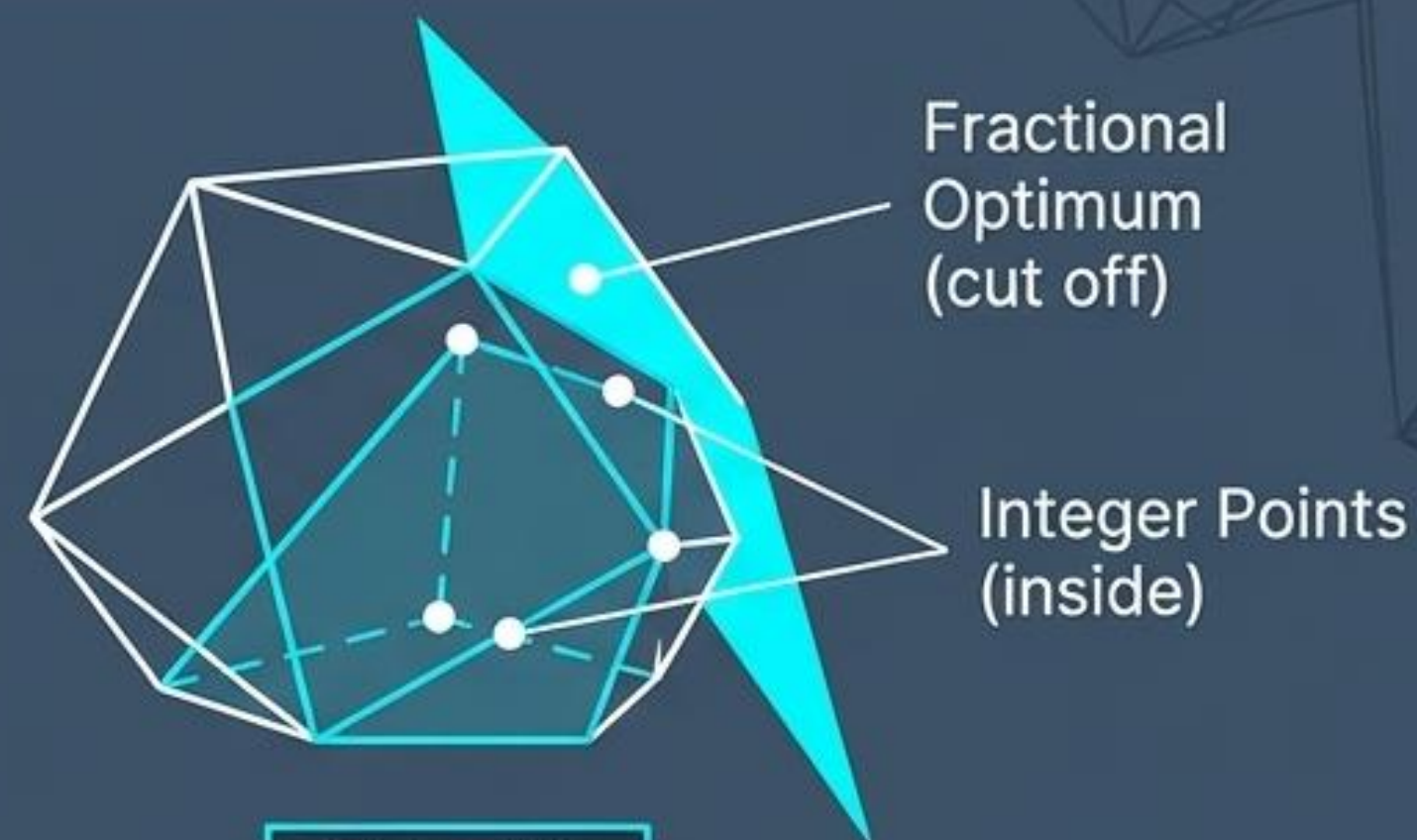
A linear inequality added to an LP relaxation that is valid for all integer feasible solutions but violated by the current fractional LP solution.

Valid Inequality

Satisfied by all integer solutions, excluding fractional ones.

Impact Box

Tightens LP relaxations, reduces the integrality gap, and drastically lowers the number of B&B nodes explored.



Motivation: While Branch-and-Cut dynamically adds constraints (rows), certain models possess exponentially many variables, requiring dynamic column addition.

[8. Column Generation and Branch-and-Price]

Core Engines

- **Column Generation:** Algorithm for large LPs starting with a subset of variables; new columns with negative reduced cost are generated iteratively.
- **The Pricing Problem:** The subproblem that uses current dual prices of the restricted master to search for a new variable with negative reduced cost.

Column Generation solves continuous LP relaxations iteratively.

Branch-and-Price integrates column generation directly into a Branch-and-Bound framework to solve integer programs.

Common Applications



Vehicle routing with time windows



Crew scheduling



Cutting stock



Bin packing

Motivation: Having traversed from complete enumeration to the most advanced integrated frameworks, we arrive at our architectural synthesis.

Summary and Key Insights

The Baseline

Exact algorithms guarantee provable optimality but battle **exponential complexity**, requiring strategic mitigation beyond Brute Force.

Memory vs. Spatial Search

- **Dynamic Programming** excels by exploiting **optimal substructure** and overlapping states.
- **Branch-and-Bound** leverages tight **LP relaxations** and bounding to **prune** vast regions of the search tree.

Logic vs. Algebra

Constraint Programming offers unmatched efficiency for complex logical relations via **domain reduction**, distinct from MIP's **algebraic bounding**.

Scaling to Massive Models

- **Branch-and-Cut** dynamically tightens bounds via **valid inequalities** (rows).
- **Branch-and-Price** efficiently navigates exponentially large **variable spaces** (columns).

Mastery of combinatorial optimization lies in selecting and combining the exact architectural framework tailored to a problem's specific structural properties.